

HOSPITAL CLINICAL DATABASE

Database Schema Documentation

100/100 Quality
Score

Grade A Schema
Grade

35 Tables Generated

36 Rules Applied

Generated On	September 05, 2026 at 11:04
Domain	Healthcare
Scale	Enterprise
GST Compliance	No
AI Provider	Together — openai/gpt-oss-120b

1. PROJECT OVERVIEW

Enable comprehensive, compliant clinical data management for a 300-bed multi-speciality hospital.

Modules

1. Infrastructure	System registry, unique ID mapping, and configuration
`unique_id_header_all`	Centralised business ID registry for tracking every entity across all modules
`factory_reset_all`	Global platform feature flags and kill-switches singleton
2. Clinical Core	Core patient-centric entities and clinical workflows (records, medication orders, lab results).
`patient_record_header_all`	Master record containing the current clinical data for a patient encounter
`patient_record_archive_all`	Historical archive mirror of patient_record_header_all
`patient_record_life_cycle_all`	State transition log and audit trail for patient_record_header_all
`patient_record_version_details_all`	Immutable version of a patient record created on each create/update/delete operation
`medication_order_header_all`	Order for a medication prescribed to a patient, includes dosage, route, frequency and status
`medication_order_archive_all`	Historical archive mirror of medication_order_header_all
`medication_order_life_cycle_all`	State transition log and audit trail for medication_order_header_all
`medication_order_status_all`	Lookup table for possible statuses of a medication order (e.g., Draft, PendingApproval, Approved, Rejected, Cancelled)
`lab_result_header_all`	Result of a laboratory test linked to a patient encounter, includes measured values and reference ranges
`lab_result_archive_all`	Historical archive mirror of lab_result_header_all
`lab_result_status_all`	Lookup for lab result processing state (Pending, Completed, Critical, Reviewed)
`approval_task_details_all`	Individual approval step in a multi-stage approval workflow for medication orders
`approval_task_life_cycle_all`	State transition log and audit trail for approval_task_details_all
`approval_task_status_all`	Lookup for the state of an approval task (Pending, Approved, Rejected, Escalated)
3. Security & RBAC	Role-based access control, permission management and runtime access decisions.
`role_definition_header_all`	Definition of a security role and the set of permissions it grants
`role_definition_archive_all`	Historical archive mirror of role_definition_header_all
`permission_all`	Atomic permission that can be assigned to a role (e.g., READ_PATIENT_RECORD, CREATE_MEDICATION_ORDER)

`rbac_store_configuration_all`	Persistent store of role-to-permission mappings used by the RBACEngine
`user_role_assignment_all`	Junction table linking users to role definitions
`access_decision_transaction_all`	Result of an RBAC evaluation for a specific data request (allow/deny, evaluated permissions)
4. Audit & Compliance Immutable audit logging, state change tracking and compliance reporting.	
`audit_log_transaction_all`	High-level audit log entry referencing detailed immutable audit records
`audit_log_entry_details_all`	Immutable detailed audit record containing before/after values, user, role, timestamp, operation type, and entity reference
`audit_log_store_configuration_all`	Tamper-evident storage location for audit log entries, indexed for search
`data_operation_transaction_all`	Event representing a read, write, update or delete request intercepted after RBAC approval
5. Billing & Ledger Charge item generation, double-entry accounting and financial posting.	
`charge_item_details_all`	Individual billable service or product generated for a patient (e.g., room charge, medication, lab test)
`ledger_transaction_transaction_all`	Double-entry accounting transaction posted by the LedgerEngine for billing, insurance, or inventory events
`ledger_account_header_all`	Chart of accounts entry used by LedgerEngine to classify debit and credit lines
`ledger_account_archive_all`	Historical archive mirror of ledger_account_header_all
6. Notification Engine Generation and delivery of system notifications to users and external systems.	
`notification_transaction_all`	Message generated by NotificationEngine for confirmations, alerts, or status updates
`notification_type_all`	Lookup for categories of notifications (e.g., Confirmation, Alert, BillingUpdate, ComplianceAlert)

Quality Validation

Score	100/100
Grade	A
Status	■ PASSED
Total Issues	0
Tables Found	35

2. TABLE-BY-TABLE DOCUMENTATION

For each table: purpose, column definitions, when to INSERT, when to UPDATE, and relationships.

1. `unique\_id\_header\_all`

Central registry for all business IDs. One row per entity type.

Column	Type	Notes
`id`	INT	System surrogate key
`table_name`	VARCHAR(100)	Logical table this ID sequence is for
`id_for`	VARCHAR(50)	Business entity this ID represents
`prefix`	VARCHAR(20)	Business ID prefix (e.g. PAT, DOC, WRD)
`last_id`	VARCHAR(15)	Last issued numeric sequence
`business_id`	VARCHAR(20)	Human-readable business ID (e.g. UI_id)
`domain_code`	VARCHAR(10)	Domain specific code (e.g. HCS for healthcare)
`effective_date`	DATETIME	Date from which this ID is valid
`expiry_date`	DATETIME	Date after which this ID is retired
`amount_limit`	DECIMAL(10,2)	Monetary ceiling associated with the ID
`flag_active`	TINYINT(1)	1=active, 0=inactive flag for quick checks
`total_child_records`	INT	Aggregate counter of all child rows linked to this entity
`count_active_children`	INT	Counter of currently active child records
`last_child_id`	VARCHAR(20)	Business ID of the most recently created child record
`description`	VARCHAR(255)	Free-form description or notes
`added_by`	INT	User ID who created this registry entry
`added_by_ref_id`	INT	FK to user_header_all.id
`status`	INT	1=active, 2=inactive, 3=deleted
`created_on`	DATETIME	Record creation timestamp
`modified_on`	DATETIME	Record modification timestamp

■ INSERT when

Once per entity type during initial setup. Example: register 'student_header_all' with prefix 'STU-'.

■ UPDATE when

Increment last_id and modified_on every time a new business ID is generated.

2. `factory\_reset\_all`

Supporting table: factory_reset_all

Column	Type	Notes
`id`	INT	System surrogate key (singleton row)
`status`	INT	1=active, 2=maintenance, 3=decommissioned
`app_version`	VARCHAR(20)	Current deployed application version
`maintenance_mode`	TINYINT(1)	1=maintenance window active, 0=normal operation
`api_throttle_enabled`	TINYINT(1)	1=API throttling enforced, 0=disabled
`feature_prescription_enabled`	TINYINT(1)	1=Prescription module enabled, 0=disabled
`feature_lab_results_enabled`	TINYINT(1)	1=Lab results module enabled, 0=disabled
`max_concurrent_jobs`	INT	Maximum number of concurrent background jobs
`default_locale`	VARCHAR(10)	Default locale for the platform
`data_retention_days`	INT	Number of days to retain transactional data
`audit_logging_enabled`	TINYINT(1)	1=Audit logging active, 0=inactive
`emergency_shutdown`	TINYINT(1)	1=Emergency shutdown engaged, 0=normal
`added_by`	INT	User ID who last modified the feature flags
`added_by_ref_id`	INT	FK to user_header_all.id
`created_on`	DATETIME	Record creation timestamp
`modified_on`	DATETIME	Record modification timestamp

■ **INSERT when**

When the corresponding business event occurs.

■ **UPDATE when**

When the record status or key fields change.

3. `patient\_record\_header\_all`

Master entity table for Patient Record records. One row per patient record entity.

Column	Type	Notes
`id`	INT	System surrogate key
`status`	INT	1=active, 2=inactive, 3=discharged, 4=deleted
`created_on`	DATETIME	Record creation timestamp
`modified_on`	DATETIME	Record modification timestamp
`PR_id`	VARCHAR(20)	Human-readable business ID (e.g. PR00001)
`patient_id`	VARCHAR(20)	Business ID of the patient
`patient_ref_id`	INT	FK to patient_header_all.id

`encounter_date`	DATETIME	Date and time of patient admission/encounter start
`discharge_date`	DATETIME	Date and time of patient discharge, if applicable
`diagnosis_code`	VARCHAR(10)	Primary diagnosis ICD-10 code
`ward_id`	INT	Ward where patient is admitted
`ward_ref_id`	INT	FK to ward_header_all.id
`bed_number`	VARCHAR(5)	Bed identifier within the ward
`total_medications`	INT	Aggregate counter of medication items linked to this record
`total_lab_tests`	INT	Aggregate counter of lab test results linked to this record
`last_medication_on`	DATETIME	Timestamp of the most recent medication order
`last_lab_result_on`	DATETIME	Timestamp of the most recent lab result entry
`is_critical`	TINYINT	Flag indicating critical condition (1=Yes, 0=No)
`insurance_provider`	VARCHAR(100)	Name of the insurance provider
`insurance_policy_no`	VARCHAR(50)	Policy number of the patient
`total_charges`	DECIMAL(12,2)	Total billed amount for the encounter
`added_by`	INT	User ID who created this patient record
`added_by_ref_id`	INT	FK to user_header_all.id

■ INSERT when	When a new patient record is registered or onboarded into the system.
■ UPDATE when	When patient record profile information changes. Always copy old row to patient_record_archive_all first.

4. `patient\_record\_archive\_all`

Historical mirror of patient_record_header_all. Stores a full copy of every row before it was changed.

Column	Type	Notes
`id`	INT	System surrogate key
`status`	INT	1=active, 2=inactive, 3=discharged, 4=deleted
`created_on`	DATETIME	Record creation timestamp
`modified_on`	DATETIME	Last modification timestamp
`PR_id`	VARCHAR(20)	Human-readable business ID (e.g. PR00001)
`patient_id`	VARCHAR(20)	Business ID of the patient

`encounter_date`	DATETIME	Date and time of patient admission/encounter start
`discharge_date`	DATETIME	Date and time of patient discharge
`diagnosis_code`	VARCHAR(10)	Primary diagnosis ICD-10 code
`ward_id`	INT	Ward where patient was admitted
`bed_number`	VARCHAR(5)	Bed identifier within the ward
`total_medications`	INT	Aggregate counter of medication items
`total_lab_tests`	INT	Aggregate counter of lab test results
`last_medication_on`	DATETIME	Timestamp of the most recent medication order
`last_lab_result_on`	DATETIME	Timestamp of the most recent lab result entry
`is_critical`	TINYINT	Flag indicating critical condition (1=Yes, 0=No)
`insurance_provider`	VARCHAR(100)	Name of the insurance provider
`insurance_policy_no`	VARCHAR(50)	Policy number of the patient
`total_charges`	DECIMAL(12,2)	Total billed amount for the encounter
`added_by`	INT	User ID who originally created the record
`archived_on`	DATETIME	When this version was archived
`archived_by`	INT	User ID who performed the archive
`archive_reason`	VARCHAR(255)	Reason for archiving this version
■ INSERT when	Every time a row in patient_record_header_all is about to be UPDATED. Copy the old row here first.	
■ UPDATE when	Never update archive rows — they are immutable historical records.	

5. `patient\_record\_life\_cycle\_all`

Status change audit trail for patient_record_header_all. Records every status transition with timestamp.

Column	Type	Notes
`id`	INT	System surrogate key
`PR_id`	VARCHAR(20)	Business ID of the patient record
`previous_status`	INT	Status before this change (1=active,2=inactive,3=discharged,4=deleted)
`new_status`	INT	Status after this change (1=active,2=inactive,3=discharged,4=deleted)
`reason`	VARCHAR(500)	Reason for status change
`changed_by`	INT	User ID who performed the transition
`changed_on`	DATETIME	Timestamp of the transition

`trigger_event`	VARCHAR(100)	What triggered this transition
`remarks`	TEXT	Additional remarks
`status`	INT	Record status (1=active,2=inactive,3=deleted)
`created_on`	DATETIME	Record creation timestamp
`modified_on`	DATETIME	Last modification timestamp

■ INSERT when	Every time the status column in patient_record_header_all changes. Record previous_status, new_status, and datetime.
■ UPDATE when	Never update life cycle rows — append only.

6. `patient\_record\_version\_details\_all`

Supporting table: patient_record_version_details_all

Column	Type	Notes
`id`	INT	System surrogate key
`status`	INT	1=active, 2=inactive, 3=archived
`created_on`	DATETIME	Record creation timestamp
`modified_on`	DATETIME	Last modification timestamp
`PR_id`	VARCHAR(20)	Business ID of the parent patient record
`sort_order`	INT	Line■item sequence within the version snapshot
`field_name`	VARCHAR(50)	Name of the field being versioned
`old_value`	VARCHAR(255)	Previous value of the field
`new_value`	VARCHAR(255)	New value of the field after change
`changed_by`	INT	User ID who made the change
`changed_on`	DATETIME	When the change occurred
`version_type`	VARCHAR(20)	CREATE, UPDATE or DELETE operation type
`remarks`	TEXT	Optional free■form notes about the change

■ INSERT when	When the corresponding business event occurs.
■ UPDATE when	When the record status or key fields change.

7. `medication\_order\_header\_all`

Master entity table for Medication Order records. One row per medication order entity.

Column	Type	Notes
`id`	INT	System surrogate key

`medication_order_id`	VARCHAR(20)	Human-readable business ID (e.g. MO00001)
`patient_id`	VARCHAR(20)	Business ID of the patient
`patient_ref_id`	INT	FK to patient_record_header_all.id
`prescribing_doctor_id`	INT	User ID of the doctor who prescribed
`prescribing_doctor_ref_id`	INT	FK to user_header_all.id
`medication_code`	VARCHAR(30)	Standard medication code (e.g. ATC, RxNorm)
`medication_name`	VARCHAR(255)	Full name of the medication
`dosage_amount`	DECIMAL(10,2)	Dosage amount per administration
`dosage_unit`	VARCHAR(20)	Unit of dosage (e.g. mg, ml, tablets)
`frequency_per_day`	INT	Number of administrations per day
`route`	VARCHAR(50)	Route of administration (e.g. oral, IV, IM)
`start_date`	DATETIME	When the medication order becomes effective
`end_date`	DATETIME	When the medication order expires or is discontinued
`instructions`	TEXT	Free-text clinical instructions for the order
`is_prn`	TINYINT	Flag indicating PRN (as needed) order
`total_items`	INT	Aggregate counter: total medication items prescribed
`count_items`	INT	Aggregate counter: number of distinct medication items
`last_item_added_on`	DATETIME	Timestamp of the most recent item addition
`added_by`	INT	User ID who created this order
`added_by_ref_id`	INT	FK to user_header_all.id
`status`	INT	1=Draft, 2=PendingApproval, 3=Approved, 4=Rejected, 5=Cancelled, 6=Deleted
`created_on`	DATETIME	Record creation timestamp
`modified_on`	DATETIME	Last modification timestamp

■ INSERT when	When a new medication order is registered or onboarded into the system.
■ UPDATE when	When medication order profile information changes. Always copy old row to medication_order_archive_all first.

8. `medication\_order\_archive\_all`

Historical mirror of medication_order_header_all. Stores a full copy of every row before it was changed.

Column	Type	Notes
`id`	INT	System surrogate key

`medication_order_id`	VARCHAR(20)	Business ID of the medication order
`patient_id`	VARCHAR(20)	Business ID of the patient
`prescribing_doctor_id`	INT	User ID of prescribing doctor
`medication_code`	VARCHAR(30)	Medication code
`medication_name`	VARCHAR(255)	Medication name
`dosage_amount`	DECIMAL(10,2)	Dosage amount
`dosage_unit`	VARCHAR(20)	Dosage unit
`frequency_per_day`	INT	Frequency per day
`route`	VARCHAR(50)	Administration route
`start_date`	DATETIME	Order start datetime
`end_date`	DATETIME	Order end datetime
`instructions`	TEXT	Clinical instructions
`is_prn`	TINYINT	PRN flag
`total_items`	INT	Total items at time of archive
`count_items`	INT	Item count at time of archive
`last_item_added_on`	DATETIME	Timestamp of last item addition at archive time
`added_by`	INT	Creator user ID
`status`	INT	Status at time of archive
`created_on`	DATETIME	Original creation timestamp
`modified_on`	DATETIME	Original last modification timestamp
`archived_on`	DATETIME	When this version was archived
`archived_by`	INT	User ID who performed the archive
`archive_reason`	VARCHAR(255)	Reason for archiving

■ **INSERT when**

Every time a row in medication_order_header_all is about to be UPDATED. Copy the old row here first.

■ **UPDATE when**

Never update archive rows — they are immutable historical records.

9. `medication\_order\_life\_cycle\_all`

Status change audit trail for medication_order_header_all. Records every status transition with timestamp.

Column	Type	Notes
`id`	INT	System surrogate key
`medication_order_id`	VARCHAR(20)	Business ID of the medication order
`previous_status`	INT	Status before transition (1=Draft,2=Pending Approval,3=Approved,4=Rejected,5=Cancelled,6=Deleted)

`new_status`	INT	Status after transition (same code set as previous_status)
`reason`	VARCHAR(500)	Reason for status change
`changed_by`	INT	User ID who performed the change
`changed_on`	DATETIME	Timestamp of the change
`trigger_event`	VARCHAR(100)	What triggered this transition
`remarks`	TEXT	Additional remarks
`status`	INT	Record status (1=active,2=inactive,3=deleted)
`created_on`	DATETIME	Record creation timestamp
`modified_on`	DATETIME	Last modification timestamp

■ INSERT when	Every time the status column in medication_order_header_all changes. Record previous_status, new_status, and datetime.
■ UPDATE when	Never update life cycle rows — append only.

10. `medication\_order\_status\_all`

Supporting table: medication_order_status_all

Column	Type	Notes
`id`	INT	System surrogate key
`status_code`	INT	Numeric status code (e.g., 1,2,3...)
`status_name`	VARCHAR(50)	Human-readable name of the status
`description`	VARCHAR(255)	Detailed description of the status
`is_final`	TINYINT	1 if this status is terminal (no further transitions)
`sort_order`	INT	Display order for UI dropdowns
`created_by`	INT	User ID who created this lookup row
`modified_by`	INT	User ID who last modified this lookup row
`created_on`	DATETIME	Record creation timestamp
`modified_on`	DATETIME	Last modification timestamp
`status`	INT	1=active,2=inactive,3=deleted

■ INSERT when	When the corresponding business event occurs.
■ UPDATE when	When the record status or key fields change.

11. `lab\_result\_header\_all`

Master entity table for Lab Result records. One row per lab result entity.

Column	Type	Notes
`id`	INT	System surrogate key
`status`	INT	1=active, 2=inactive, 3=deleted, 4=archived
`created_on`	DATETIME	Record creation timestamp
`modified_on`	DATETIME	Last modification timestamp
`lab_result_id`	VARCHAR(20)	Human-readable business ID (e.g. LR-00001)
`patient_record_id`	VARCHAR(20)	Business ID of the patient record
`patient_record_ref_id`	INT	FK to patient_record_header_all.id
`encounter_id`	VARCHAR(20)	Business ID of the patient encounter
`medication_order_id`	VARCHAR(20)	Business ID of related medication order, if any
`medication_order_ref_id`	INT	FK to medication_order_header_all.id
`test_code`	VARCHAR(20)	Lab test code (e.g. CBC, LFT)
`test_name`	VARCHAR(100)	Descriptive name of the lab test
`result_value`	VARCHAR(100)	Raw result as reported by the lab
`result_numeric`	DECIMAL(10,2)	Numeric representation of the result, if applicable
`unit`	VARCHAR(20)	Unit of measurement for numeric result
`reference_range_low`	DECIMAL(10,2)	Lower bound of reference range
`reference_range_high`	DECIMAL(10,2)	Upper bound of reference range
`result_flag`	VARCHAR(20)	Flag indicating result status (e.g. High, Low, Normal)
`result_json`	JSON	Structured JSON payload for complex result data
`collected_on`	DATETIME	Date-time when specimen was collected
`reported_on`	DATETIME	Date-time when result was reported
`performed_by`	INT	User ID of the lab technician who performed the test
`performed_by_ref_id`	INT	FK to user_header_all.id
`added_by`	INT	User ID who created this lab result record
`added_by_ref_id`	INT	FK to user_header_all.id
`total_items`	INT	Aggregate counter: total number of result items (usually 1)
`count_abnormal`	INT	Aggregate counter: number of abnormal findings in this result set
`last_result_on`	DATETIME	Timestamp of the most recent result update
`notes`	TEXT	Additional clinical notes or comments

■ INSERT when

When a new lab result is registered or onboarded into the system.

■ UPDATE when

When lab result profile information changes. Always copy old row to lab_result_archive_all first.

12. `lab\_result\_archive\_all`

Historical mirror of lab_result_header_all. Stores a full copy of every row before it was changed.

Column	Type	Notes
`id`	INT	System surrogate key
`status`	INT	Historical status snapshot (mirrors source status)
`created_on`	DATETIME	Original creation timestamp of the source record
`modified_on`	DATETIME	Original last modification timestamp of the source record
`lab_result_id`	VARCHAR(20)	Business ID of the lab result
`patient_record_id`	VARCHAR(20)	Business ID of the patient record
`encounter_id`	VARCHAR(20)	Business ID of the patient encounter
`medication_order_id`	VARCHAR(20)	Business ID of related medication order
`test_code`	VARCHAR(20)	Lab test code
`test_name`	VARCHAR(100)	Lab test name
`result_value`	VARCHAR(100)	Raw result value
`result_numeric`	DECIMAL(10,2)	Numeric result, if applicable
`unit`	VARCHAR(20)	Result unit
`reference_range_low`	DECIMAL(10,2)	Reference range low
`reference_range_high`	DECIMAL(10,2)	Reference range high
`result_flag`	VARCHAR(20)	Result flag (High/Low/Normal)
`result_json`	JSON	JSON payload of result
`collected_on`	DATETIME	Specimen collection timestamp
`reported_on`	DATETIME	Result reporting timestamp
`performed_by`	INT	Lab technician user ID
`added_by`	INT	User who originally added the record
`total_items`	INT	Total items counter at time of archiving
`count_abnormal`	INT	Abnormal count at time of archiving
`last_result_on`	DATETIME	Timestamp of last result update
`notes`	TEXT	Clinical notes
`archived_on`	DATETIME	When this version was archived
`archived_by`	INT	User who performed the archiving
`archive_reason`	VARCHAR(255)	Reason for archiving this version

■ INSERT when	Every time a row in lab_result_header_all is about to be UPDATED. Copy the old row here first.
■ UPDATE when	Never update archive rows — they are immutable historical records.

13. `lab\_result\_status\_all`

Supporting table: lab_result_status_all

Column	Type	Notes
`id`	INT	System surrogate key
`status_code`	VARCHAR(20)	Short code for status (e.g. PENDING, COMPLETED)
`description`	VARCHAR(255)	Full description of the status
`is_critical`	TINYINT	1=critical result, 0=non-critical
`notification_required`	TINYINT	1=notify clinician, 0=no notification
`sort_order`	INT	Display order for UI dropdowns
`color_code`	VARCHAR(7)	Hex color code for UI representation
`allowed_transition`	VARCHAR(255)	Comma-separated list of status_codes allowed to transition from this state
`created_on`	DATETIME	Record creation timestamp
`modified_on`	DATETIME	Last modification timestamp
`added_by`	INT	User who added the status record
`status`	INT	1=active, 2=inactive, 3=deleted

■ INSERT when	When the corresponding business event occurs.
■ UPDATE when	When the record status or key fields change.

14. `approval\_task\_details\_all`

Supporting table: approval_task_details_all

Column	Type	Notes
`id`	INT	System surrogate key
`status`	INT	1=active, 2=inactive, 3=deleted, 4=revoked
`created_on`	DATETIME	Record creation timestamp
`modified_on`	DATETIME	Last modification timestamp
`medication_order_id`	VARCHAR(20)	Business ID of the parent medication order
`medication_order_ref_id`	INT	FK to medication_order_header_all.id
`sort_order`	INT	Sequence number of this approval step

`approver_id`	INT	User ID of the approver for this step
`approver_ref_id`	INT	FK to user_header_all.id
`approval_status`	INT	1=Pending, 2=Approved, 3=Rejected, 4=Escalated
`approval_comment`	TEXT	Approver comments for this step
`approved_on`	DATETIME	Timestamp when approval decision was made
`reviewed_by`	INT	User ID who reviewed the approval after decision
`reviewed_by_ref_id`	INT	FK to user_header_all.id
`review_on`	DATETIME	Timestamp of the review
`added_by`	INT	User who created this approval task record
`added_by_ref_id`	INT	FK to user_header_all.id
`notes`	TEXT	Additional notes for the approval step

■ INSERT when	When the corresponding business event occurs.
■ UPDATE when	When the record status or key fields change.

15. `approval\_task\_life\_cycle\_all`

Status change audit trail for approval_task_header_all. Records every status transition with timestamp.

Column	Type	Notes
`id`	INT	System surrogate key
`approval_task_id`	VARCHAR(20)	Business ID of the approval task (e.g. ATK-00001)
`approval_task_ref_id`	INT	FK to approval_task_details_all.id
`previous_status`	INT	Previous state of the task (1=Pending,2=Approved,3=Rejected,4=Escalated,5=Cancelled)
`new_status`	INT	New state of the task (1=Pending,2=Approved,3=Rejected,4=Escalated,5=Cancelled)
`reason`	VARCHAR(500)	Reason for status change
`changed_by`	INT	User ID who performed the transition
`changed_by_ref_id`	INT	FK to user_header_all.id
`changed_on`	DATETIME	Timestamp when the transition occurred
`trigger_event`	VARCHAR(100)	What triggered this transition (e.g., auto_escalation, manual_approval)
`remarks`	TEXT	Additional remarks or comments
`status`	INT	1=active, 2=inactive, 3=deleted
`created_on`	DATETIME	Record creation timestamp

<code>`modified_on`</code>	DATETIME	Last modification timestamp
■ INSERT when	Every time the status column in <code>approval_task_header_all</code> changes. Record <code>previous_status</code> , <code>new_status</code> , and <code>datetime</code> .	
■ UPDATE when	Never update life cycle rows — append only.	

16. ``approval_task_status_all``

Supporting table: `approval_task_status_all`

Column	Type	Notes
<code>`id`</code>	INT	System surrogate key
<code>`status_code`</code>	INT	1=Pending,2=Approved,3=Rejected,4=Escalated,5=Cancelled
<code>`status_name`</code>	VARCHAR(50)	Human-readable name of the status
<code>`description`</code>	TEXT	Detailed description of what the status means
<code>`is_final`</code>	TINYINT(1)	1=final state, 0=intermediate state
<code>`escalation_level`</code>	INT	Numeric level of escalation required (if any)
<code>`notify_role`</code>	VARCHAR(50)	System role to be notified when entering this status
<code>`requires_approval`</code>	TINYINT(1)	1=requires explicit approval, 0=auto-transition
<code>`auto_transition_to`</code>	INT	Status code to auto-move to after timeout (if applicable)
<code>`timeout_minutes`</code>	INT	Minutes after which auto-transition occurs
<code>`created_by`</code>	INT	User ID who created this status record
<code>`modified_by`</code>	INT	User ID who last modified this status record
<code>`created_on`</code>	DATETIME	Record creation timestamp
<code>`modified_on`</code>	DATETIME	Last modification timestamp
<code>`status`</code>	INT	1=active, 2=inactive, 3=deleted
■ INSERT when	When the corresponding business event occurs.	
■ UPDATE when	When the record status or key fields change.	

17. ``role_definition_header_all``

Master entity table for Role Definition records. One row per role definition entity.

Column	Type	Notes
<code>`id`</code>	INT	System surrogate key
<code>`status`</code>	INT	1=active, 2=inactive, 3=deleted, 4=archived

`created_on`	DATETIME	Record creation timestamp
`modified_on`	DATETIME	Record modification timestamp
`role_id`	VARCHAR(20)	Human-readable business ID for role (e.g. RD-00001)
`role_name`	VARCHAR(100)	Descriptive name of the security role
`role_description`	VARCHAR(500)	Detailed description of role responsibilities
`is_system_role`	TINYINT	1=system-defined role, 0=custom role
`department_id`	INT	FK to department_header_all (if role scoped to a department)
`department_ref_id`	INT	FK to department_header_all.id
`effective_date`	DATE	Date from which the role becomes effective
`expiry_date`	DATE	Date after which the role is no longer valid
`total_permissions`	INT	Aggregate counter of permissions assigned to this role
`total_users`	INT	Aggregate counter of users assigned to this role
`last_permission_assigned_on`	DATETIME	Timestamp of most recent permission assignment
`last_user_assigned_on`	DATETIME	Timestamp of most recent user assignment
`added_by`	INT	User ID who created this role record
`added_by_ref_id`	INT	FK to user_header_all.id

■ INSERT when	When a new role definition is registered or onboarded into the system.
■ UPDATE when	When role definition profile information changes. Always copy old row to role_definition_archive_all first.

18. `role\_definition\_archive\_all`

Historical mirror of role_definition_header_all. Stores a full copy of every row before it was changed.

Column	Type	Notes
`id`	INT	System surrogate key
`status`	INT	1=active, 2=inactive, 3=deleted, 4=archived
`created_on`	DATETIME	Record creation timestamp
`modified_on`	DATETIME	Last modification timestamp
`role_id`	VARCHAR(20)	Human-readable business ID for role (e.g. RD-00001)
`role_name`	VARCHAR(100)	Descriptive name of the security role
`role_description`	VARCHAR(500)	Detailed description of role responsibilities
`is_system_role`	TINYINT	1=system-defined role, 0=custom role

<code>`department_id`</code>	INT	FK to department_header_all (if role scoped to a department)
<code>`effective_date`</code>	DATE	Date from which the role became effective
<code>`expiry_date`</code>	DATE	Date after which the role expired
<code>`total_permissions`</code>	INT	Aggregate counter of permissions at time of archiving
<code>`total_users`</code>	INT	Aggregate counter of users at time of archiving
<code>`last_permission_assigned_on`</code>	DATETIME	Timestamp of most recent permission assignment before archive
<code>`last_user_assigned_on`</code>	DATETIME	Timestamp of most recent user assignment before archive
<code>`added_by`</code>	INT	User ID who originally created the role
<code>`archived_on`</code>	DATETIME	When this version was archived
<code>`archived_by`</code>	INT	User ID who performed the archiving
<code>`archive_reason`</code>	VARCHAR(255)	Reason for archiving this version
■ INSERT when	Every time a row in role_definition_header_all is about to be UPDATED. Copy the old row here first.	
■ UPDATE when	Never update archive rows — they are immutable historical records.	

19. ``permission_all``

Supporting table: permission_all

Column	Type	Notes
<code>`id`</code>	INT	System surrogate key
<code>`status`</code>	INT	1=active, 2=inactive, 3=deprecated
<code>`created_on`</code>	DATETIME	Record creation timestamp
<code>`modified_on`</code>	DATETIME	Record modification timestamp
<code>`permission_code`</code>	VARCHAR(50)	Machine■readable code (e.g. READ_PATIENT_RECORD)
<code>`permission_name`</code>	VARCHAR(100)	Human■readable name of the permission
<code>`permission_description`</code>	VARCHAR(500)	Detailed description of what the permission allows
<code>`module_name`</code>	VARCHAR(100)	Application module to which this permission belongs (e.g. PatientCare, Billing)
<code>`is_sensitive`</code>	TINYINT	1=permission grants access to sensitive data, 0=regular
<code>`created_by`</code>	INT	User ID who created the permission record
<code>`created_by_ref_id`</code>	INT	FK to user_header_all.id

`modified_by`	INT	User ID who last modified the permission record
`modified_by_ref_id`	INT	FK to user_header_all.id
■ INSERT when	When the corresponding business event occurs.	
■ UPDATE when	When the record status or key fields change.	

20. `rbac\_store\_configuration\_all`

Temporal configuration for rbac_store. Stores settings with effective date ranges.

Column	Type	Notes
`id`	INT	System surrogate key
`status`	INT	1=active, 2=inactive, 3=revoked
`created_on`	DATETIME	Record creation timestamp
`modified_on`	DATETIME	Record modification timestamp
`role_id`	VARCHAR(20)	Business ID of role
`role_ref_id`	INT	FK to role_definition_header_all.id
`permission_id`	INT	ID of permission
`permission_ref_id`	INT	FK to permission_all.id
`entity_type`	VARCHAR(50)	Type of entity the mapping applies to (e.g. PATIENT_RECORD, LAB_RESULT)
`entity_id`	VARCHAR(20)	Specific entity identifier if mapping is scoped to a single entity
`effective_from`	DATE	Date from which this mapping becomes effective
`effective_to`	DATE	Date after which this mapping is no longer effective
`config_note`	VARCHAR(255)	Additional notes or business rules for this mapping
`updated_by`	INT	User ID who performed the latest update
`updated_by_ref_id`	INT	FK to user_header_all.id
`approved_by`	INT	User ID who approved the configuration change
`approved_by_ref_id`	INT	FK to user_header_all.id
■ INSERT when	When a new configuration is set. Insert new row with new from_date. Do not update the old row — let it expire.	
■ UPDATE when	Update status to 0 (past) when a newer configuration takes over.	

21. `user\_role\_assignment\_all`

Supporting table: user_role_assignment_all

Column	Type	Notes
`id`	INT	System surrogate key
`status`	INT	1=active, 2=inactive, 3=revoked
`created_on`	DATETIME	Record creation timestamp
`modified_on`	DATETIME	Record modification timestamp
`user_id`	VARCHAR(20)	Human-readable business ID of the user (e.g. USR00001)
`user_ref_id`	INT	FK to user_header_all.id
`role_id`	VARCHAR(20)	Human-readable business ID of the role (e.g. ROLEADMIN)
`role_ref_id`	INT	FK to role_definition_header_all.id
`assigned_by`	INT	User ID who performed the assignment
`assigned_by_ref_id`	INT	FK to user_header_all.id
`assigned_on`	DATETIME	Timestamp when the role was assigned
`assignment_reason`	VARCHAR(500)	Reason for assigning this role to the user
`expires_on`	DATETIME	Optional expiry timestamp for temporary assignments
`notes`	TEXT	Additional free-form notes about the assignment
`is_primary`	TINYINT(1)	1=primary role for the user, 0=secondary/auxiliary

■ INSERT when

When the corresponding business event occurs.

■ UPDATE when

When the record status or key fields change.

22. `access\_decision\_transaction\_all`

Event/ledger table for access_decision operations. Append-only record of all events.

Column	Type	Notes
`id`	INT	System surrogate key
`status`	INT	1=recorded, 2=archived, 3=invalid
`created_on`	DATETIME	Record creation timestamp
`modified_on`	DATETIME	Record modification timestamp
`month_year`	VARCHAR(7)	YYYYMM for partitioning
`transaction_ref`	VARCHAR(30)	Human-readable transaction ID
`requestor_user_id`	VARCHAR(20)	Business ID of the user who initiated the request
`requestor_user_ref_id`	INT	FK to user_header_all.id

`target_user_id`	VARCHAR(20)	Business ID of the user whose data is being accessed
`target_user_ref_id`	INT	FK to user_header_all.id
`resource_type`	VARCHAR(50)	Type of resource accessed (e.g. PATIENT_RECORD, LAB_RESULT)
`resource_id`	VARCHAR(30)	Business ID of the specific resource instance
`action`	VARCHAR(30)	Requested action (e.g. READ, UPDATE, DELETE)
`decision`	VARCHAR(10)	Result of RBAC evaluation: ALLOW or DENY
`evaluated_permissions`	JSON	List of permissions evaluated during the decision
`before_state`	JSON	Snapshot of the resource state before the action (if applicable)
`after_state`	JSON	Snapshot of the resource state after the action (if applicable)
`amount`	DECIMAL(10,2)	Financial amount involved in the request, if any
`added_by`	INT	User ID who logged this decision record
`added_by_ref_id`	INT	FK to user_header_all.id
`device_id`	VARCHAR(50)	Identifier of the device used for the request
`ip_address`	VARCHAR(45)	IP address of the requester
■ INSERT when	Every time a new transaction, payment, or event occurs. Each row represents one atomic event.	
■ UPDATE when	Update status column only (e.g. pending → paid). Never modify financial columns after insert.	

23. `audit\_log\_transaction\_all`

Event/ledger table for audit_log operations. Append-only record of all events.

Column	Type	Notes
`id`	INT	System surrogate key
`status`	INT	1=active, 2=inactive, 3=deleted
`created_on`	DATETIME	Record creation timestamp
`modified_on`	DATETIME	Record modification timestamp
`month_year`	VARCHAR(7)	YYYY-MM for partitioning
`transaction_ref`	VARCHAR(30)	Human-readable transaction ID
`table_name`	VARCHAR(100)	Name of the table being audited

`row_id`	VARCHAR(36)	Primary key of the audited row (UUID or business ID)
`action`	VARCHAR(10)	INSERT UPDATE DELETE READ
`before_state`	JSON	JSON snapshot of the row before the operation
`after_state`	JSON	JSON snapshot of the row after the operation
`actor_user_id`	INT	User ID who performed the action
`actor_user_ref_id`	INT	FK to user_header_all.id
`device_id`	VARCHAR(50)	Identifier of the device used
`ip_address`	VARCHAR(45)	IP address of the requester
`user_agent`	VARCHAR(255)	User agent string of the client
`added_by`	INT	User ID who recorded this audit entry
`added_by_ref_id`	INT	FK to user_header_all.id
`amount`	DECIMAL(10,2)	Financial amount involved, if any

■ INSERT when	Every time a new transaction, payment, or event occurs. Each row represents one atomic event.
■ UPDATE when	Update status column only (e.g. pending → paid). Never modify financial columns after insert.

24. `audit\_log\_entry\_details\_all`

Supporting table: audit_log_entry_details_all

Column	Type	Notes
`id`	INT	System surrogate key
`status`	INT	1=active, 2=inactive, 3=deleted
`created_on`	DATETIME	Record creation timestamp
`modified_on`	DATETIME	Record modification timestamp
`audit_log_transaction_id`	INT	FK to parent audit_log_transaction_all
`audit_log_transaction_ref_id`	INT	FK to audit_log_transaction_all.id
`sort_order`	INT	Line item sequence within the audit transaction
`field_name`	VARCHAR(100)	Name of the audited field
`old_value`	VARCHAR(255)	Value before change
`new_value`	VARCHAR(255)	Value after change
`changed_by`	INT	User ID who caused the change
`changed_by_ref_id`	INT	FK to user_header_all.id

<code>`changed_on`</code>	DATETIME	Timestamp of the change
■ INSERT when	When the corresponding business event occurs.	
■ UPDATE when	When the record status or key fields change.	

25. ``audit_log_store_configuration_all``

Temporal configuration for `audit_log_store`. Stores settings with effective date ranges.

Column	Type	Notes
<code>`id`</code>	INT	System surrogate key
<code>`status`</code>	INT	1=active, 2=inactive, 3=deleted
<code>`created_on`</code>	DATETIME	Record creation timestamp
<code>`modified_on`</code>	DATETIME	Record modification timestamp
<code>`entity_type`</code>	VARCHAR(50)	Type of entity the config applies to (e.g. PATIENT, LAB_RESULT)
<code>`entity_id`</code>	VARCHAR(30)	Business ID of the entity
<code>`effective_from`</code>	DATE	Configuration becomes effective on this date
<code>`effective_to`</code>	DATE	Configuration expires on this date
<code>`retention_days`</code>	INT	Number of days audit logs are retained
<code>`encryption_key_id`</code>	VARCHAR(50)	Reference to encryption key for tamper-evidence
<code>`updated_by`</code>	INT	User ID who last updated this configuration
<code>`updated_by_ref_id`</code>	INT	FK to <code>user_header_all.id</code>
<code>`approved_by`</code>	INT	User ID who approved this configuration
<code>`approved_by_ref_id`</code>	INT	FK to <code>user_header_all.id</code>
■ INSERT when	When a new configuration is set. Insert new row with new <code>from_date</code> . Do not update the old row — let it expire.	
■ UPDATE when	Update status to 0 (past) when a newer configuration takes over.	

26. ``data_operation_transaction_all``

Event/ledger table for `data_operation` operations. Append-only record of all events.

Column	Type	Notes
<code>`id`</code>	INT	System surrogate key
<code>`status`</code>	INT	1=active, 2=inactive, 3=deleted
<code>`created_on`</code>	DATETIME	Record creation timestamp
<code>`modified_on`</code>	DATETIME	Record modification timestamp

`month_year`	VARCHAR(7)	YYYY-MM for partitioning
`transaction_ref`	VARCHAR(30)	Human-readable transaction ID
`operation_type`	VARCHAR(10)	READ WRITE UPDATE DELETE
`target_table`	VARCHAR(100)	Table on which the operation was performed
`target_row_id`	VARCHAR(36)	Primary key of the target row
`before_state`	JSON	JSON snapshot before the operation (if applicable)
`after_state`	JSON	JSON snapshot after the operation (if applicable)
`actor_user_id`	INT	User ID who performed the operation
`actor_user_ref_id`	INT	FK to user_header_all.id
`device_id`	VARCHAR(50)	Identifier of the device used
`ip_address`	VARCHAR(45)	IP address of the requester
`user_agent`	VARCHAR(255)	User-agent string of the client
`added_by`	INT	User ID who recorded this data operation
`added_by_ref_id`	INT	FK to user_header_all.id
`amount`	DECIMAL(10,2)	Financial amount involved, if any

■ INSERT when	Every time a new transaction, payment, or event occurs. Each row represents one atomic event.
■ UPDATE when	Update status column only (e.g. pending → paid). Never modify financial columns after insert.

27. `charge\_item\_details\_all`

Supporting table: charge_item_details_all

Column	Type	Notes
`id`	INT	System surrogate key
`status`	INT	1=active, 2=inactive, 3=deleted
`created_on`	DATETIME	Record creation timestamp
`modified_on`	DATETIME	Record modification timestamp
`charge_item_id`	VARCHAR(20)	Business ID of the charge item (e.g. CHG-00001)
`patient_id`	VARCHAR(20)	Business ID of the patient
`patient_ref_id`	INT	FK to patient_record_header_all.id
`admission_id`	VARCHAR(20)	Business ID of the admission episode
`admission_ref_id`	INT	FK to admission_header_all.id
`service_code`	VARCHAR(30)	Code representing the billable service or product

`service_description`	VARCHAR(255)	Human readable description of the service
`quantity`	DECIMAL(10,2)	Quantity of the service rendered
`unit_price`	DECIMAL(10,2)	Price per unit of the service
`total_amount`	DECIMAL(10,2)	Quantity * unit_price
`sgst_percentage`	DECIMAL(5,2)	State GST rate applied
`sgst_amount`	DECIMAL(10,2)	State GST amount
`cgst_percentage`	DECIMAL(5,2)	Central GST rate applied
`cgst_amount`	DECIMAL(10,2)	Central GST amount
`igst_percentage`	DECIMAL(5,2)	Integrated GST rate applied (interstate)
`igst_amount`	DECIMAL(10,2)	Integrated GST amount
`sort_order`	INT	Line number / sequence for ordering within the bill
`added_by`	INT	User ID who created this charge item
`added_by_ref_id`	INT	FK to user_header_all.id
`device_id`	VARCHAR(50)	Device identifier from which the item was entered
`ip_address`	VARCHAR(45)	IP address of the client creating the record

■ **INSERT when** When the corresponding business event occurs.

■ **UPDATE when** When the record status or key fields change.

28. `ledger\_account\_header\_all`

Master entity table for Ledger Account records. One row per ledger account entity.

Column	Type	Notes
`id`	INT	System surrogate key
`status`	INT	1=active, 2=inactive, 3=closed, 4=deleted
`created_on`	DATETIME	Record creation timestamp
`modified_on`	DATETIME	Record modification timestamp
`la_id`	VARCHAR(20)	Human-readable Ledger Account ID (e.g. LA-00001)
`account_number`	VARCHAR(100)	Unique account number in the chart of accounts
`account_name`	VARCHAR(150)	Descriptive name of the ledger account
`account_type`	VARCHAR(30)	Type of account (Asset, Liability, Equity, Revenue, Expense)
`currency`	VARCHAR(3)	Currency code (ISO 4217)
`opening_balance`	DECIMAL(15,4)	Balance at the start of the accounting period
`current_balance`	DECIMAL(15,4)	Running balance after latest transaction

`parent_account_id`	VARCHAR(20)	Business ID of parent ledger account for hierarchy
`parent_account_ref_id`	INT	FK to ledger_account_header_all.id
`is_control_account`	TINYINT(1)	1=control account, 0=detail account
`total_debits`	DECIMAL(15,4)	Aggregate sum of debit postings
`total_credits`	DECIMAL(15,4)	Aggregate sum of credit postings
`transaction_count`	INT	Number of transactions posted to this account
`last_transaction_on`	DATETIME	Timestamp of the most recent transaction
`added_by`	INT	User ID who created the ledger account
`added_by_ref_id`	INT	FK to user_header_all.id
`department_code`	VARCHAR(20)	Department to which the account belongs (e.g. CARD, RAD, PHARMA)
`cost_center`	VARCHAR(20)	Cost centre identifier for internal accounting
`is_active`	TINYINT(1)	1=account is active for posting, 0=inactive

■ INSERT when

When a new ledger account is registered or onboarded into the system.

■ UPDATE when

When ledger account profile information changes. Always copy old row to ledger_account_archive_all first.

29. `ledger\_account\_archive\_all`

Historical mirror of ledger_account_header_all. Stores a full copy of every row before it was changed.

Column	Type	Notes
`id`	INT	System surrogate key
`status`	INT	Status at time of archiving
`created_on`	DATETIME	Original creation timestamp
`modified_on`	DATETIME	Original last modification timestamp
`la_id`	VARCHAR(20)	Ledger Account business ID
`account_number`	VARCHAR(100)	Account number
`account_name`	VARCHAR(150)	Account name
`account_type`	VARCHAR(30)	Account type
`currency`	VARCHAR(3)	Currency code
`opening_balance`	DECIMAL(15,4)	Opening balance at archive time
`current_balance`	DECIMAL(15,4)	Balance at archive time
`parent_account_id`	VARCHAR(20)	Parent account business ID
`is_control_account`	TINYINT(1)	Control account flag
`total_debits`	DECIMAL(15,4)	Total debits till archive

`total_credits`	DECIMAL(15,4)	Total credits till archive
`transaction_count`	INT	Number of transactions till archive
`last_transaction_on`	DATETIME	Timestamp of last transaction before archive
`added_by`	INT	User who originally added the account
`department_code`	VARCHAR(20)	Department code
`cost_center`	VARCHAR(20)	Cost centre
`is_active`	TINYINT(1)	Active flag at archive time
`archived_on`	DATETIME	When this version was archived
`archived_by`	INT	User who performed the archiving
`archive_reason`	VARCHAR(255)	Reason for archiving this version

■ INSERT when	Every time a row in ledger_account_header_all is about to be UPDATED. Copy the old row here first.
■ UPDATE when	Never update archive rows — they are immutable historical records.

30. `ledger\_transaction\_transaction\_all`

Event/ledger table for ledger_transaction operations. Append-only record of all events.

Column	Type	Notes
`id`	INT	System surrogate key
`status`	INT	1=posted, 2=failed, 3=reversed, 4=deleted
`created_on`	DATETIME	Record creation timestamp
`modified_on`	DATETIME	Record modification timestamp
`transaction_ref`	VARCHAR(30)	Human-readable transaction ID (e.g. TRX-20230905-001)
`month_year`	VARCHAR(7)	YYYYMM for partitioning and analytics
`debit_account_id`	VARCHAR(20)	Business ID of debit ledger account
`debit_account_ref_id`	INT	FK to ledger_account_header_all.id
`credit_account_id`	VARCHAR(20)	Business ID of credit ledger account
`credit_account_ref_id`	INT	FK to ledger_account_header_all.id
`amount`	DECIMAL(15,4)	Transaction amount (currency defined by accounts)
`transaction_type`	VARCHAR(50)	Type of transaction (e.g. PATIENT_CHARGE, INSURANCE_PAYMENT, INVENTORY_ADJUST)
`reference_id`	VARCHAR(30)	External reference (e.g. invoice number, claim ID)
`posting_date`	DATETIME	Date and time when transaction was posted to the ledger

`description`	TEXT	Narrative description of the transaction
`sgst_percentage`	DECIMAL(5,2)	State GST rate applied to this transaction
`sgst_amount`	DECIMAL(10,2)	State GST amount
`cgst_percentage`	DECIMAL(5,2)	Central GST rate applied to this transaction
`cgst_amount`	DECIMAL(10,2)	Central GST amount
`igst_percentage`	DECIMAL(5,2)	Integrated GST rate applied (interstate)
`igst_amount`	DECIMAL(10,2)	Integrated GST amount
`before_balance_debit`	DECIMAL(15,4)	Debit account balance before posting
`after_balance_debit`	DECIMAL(15,4)	Debit account balance after posting
`before_balance_credit`	DECIMAL(15,4)	Credit account balance before posting
`after_balance_credit`	DECIMAL(15,4)	Credit account balance after posting
`added_by`	INT	User ID who initiated the transaction
`added_by_ref_id`	INT	FK to user_header_all.id
`device_id`	VARCHAR(50)	Device identifier from which transaction was created
`ip_address`	VARCHAR(45)	IP address of the client creating the transaction

■ INSERT when

Every time a new transaction, payment, or event occurs. Each row represents one atomic event.

■ UPDATE when

Update status column only (e.g. pending → paid). Never modify financial columns after insert.

31. `notification\_transaction\_all`

Event/ledger table for notification operations. Append-only record of all events.

Column	Type	Notes
`id`	INT	System surrogate key
`status`	INT	1=active, 2=inactive, 3=deleted
`month_year`	VARCHAR(7)	YYYY-MM for partitioning
`transaction_ref`	VARCHAR(30)	Human-readable transaction ID
`notification_type_code`	VARCHAR(20)	FK to notification_type_all.type_code
`sender_user_id`	INT	User who triggered the notification
`sender_user_ref_id`	INT	FK to user_header_all.id
`recipient_user_id`	INT	User who receives the notification
`recipient_user_ref_id`	INT	FK to user_header_all.id
`patient_id`	VARCHAR(20)	Business ID of patient
`patient_ref_id`	INT	FK to patient_record_header_all.id

`old_status`	INT	Previous status value (if applicable)
`new_status`	INT	New status value (if applicable)
`amount`	DECIMAL(10,2)	Monetary amount related to the notification, if any
`sgst_amount`	DECIMAL(10,2)	State GST amount, if financial
`cgst_amount`	DECIMAL(10,2)	Central GST amount, if financial
`message_body`	TEXT	Full notification message content
`added_by`	INT	User ID who created this record
`added_by_ref_id`	INT	FK to user_header_all.id
`device_id`	VARCHAR(50)	Device identifier from which notification was generated
`ip_address`	VARCHAR(45)	IP address of the source system
`created_on`	DATETIME	Record creation timestamp
`modified_on`	DATETIME	Record modification timestamp

■ INSERT when	Every time a new transaction, payment, or event occurs. Each row represents one atomic event.
■ UPDATE when	Update status column only (e.g. pending → paid). Never modify financial columns after insert.

32. `notification\_type\_all`

Supporting table: notification_type_all

Column	Type	Notes
`id`	INT	System surrogate key
`status`	INT	1=active, 2=inactive, 3=deleted
`type_code`	VARCHAR(20)	Business code for the notification type (e.g., CONFIRM, ALERT, BILL_UPD)
`type_name`	VARCHAR(100)	Human-readable name of the notification type
`description`	TEXT	Detailed description of when this notification is used
`email_template`	VARCHAR(100)	Name of the email template associated with this type
`sms_template`	VARCHAR(100)	Name of the SMS template associated with this type
`push_template`	VARCHAR(100)	Name of the push notification template
`priority`	INT	Priority level (1=highest, 10=lowest)
`default_channel`	VARCHAR(20)	Default delivery channel (e.g., email, sms, push)

`retention_days`	INT	Number of days the notification is retained in the system
`is_auditable`	INT	1=changes are audited, 0=not audited
`requires_approval`	INT	1=requires approval workflow, 0=auto■send
`created_by`	INT	User ID who created this notification type
`modified_by`	INT	User ID who last modified this notification type
`created_on`	DATETIME	Record creation timestamp
`modified_on`	DATETIME	Last modification timestamp

■ INSERT when	When the corresponding business event occurs.
■ UPDATE when	When the record status or key fields change.

33. `lab\_result\_life\_cycle\_all`

Status change audit trail for lab_result_header_all. Records every status transition with timestamp.

Column	Type	Notes
`id`	INT	System surrogate key
`lab_result_id`	VARCHAR(20)	Business ID of the lab result
`lab_result_ref_id`	INT	FK to lab_result_header_all.id
`previous_status`	INT	Previous status code
`new_status`	INT	New status code
`reason`	VARCHAR(500)	Reason for status change
`changed_by`	INT	User ID who performed the transition
`changed_by_ref_id`	INT	FK to user_header_all.id
`changed_on`	DATETIME	Timestamp of the transition
`remarks`	TEXT	Additional remarks
`status`	INT	1=active, 2=inactive, 3=deleted
`created_on`	DATETIME	Record creation timestamp
`modified_on`	DATETIME	Record modification timestamp

■ INSERT when	Every time the status column in lab_result_header_all changes. Record previous_status, new_status, and datetime.
■ UPDATE when	Never update life cycle rows — append only.

34. `ledger\_account\_life\_cycle\_all`

Status change audit trail for ledger_account_header_all. Records every status transition with timestamp.

Column	Type	Notes
`id`	INT	System surrogate key
`ledger_account_id`	VARCHAR(20)	Business ID of the ledger account
`ledger_account_ref_id`	INT	FK to ledger_account_header_all.id
`previous_status`	INT	Previous status code
`new_status`	INT	New status code
`reason`	VARCHAR(500)	Reason for status change
`changed_by`	INT	User ID who performed the transition
`changed_by_ref_id`	INT	FK to user_header_all.id
`changed_on`	DATETIME	Timestamp of the transition
`remarks`	TEXT	Additional remarks
`status`	INT	1=active, 2=inactive, 3=deleted
`created_on`	DATETIME	Record creation timestamp
`modified_on`	DATETIME	Record modification timestamp
■ INSERT when	Every time the status column in ledger_account_header_all changes. Record previous_status, new_status, and datetime.	
■ UPDATE when	Never update life cycle rows — append only.	

35. `role\_definition\_life\_cycle\_all`

Status change audit trail for role_definition_header_all. Records every status transition with timestamp.

Column	Type	Notes
`id`	INT	System surrogate key
`role_id`	VARCHAR(20)	Business ID of the role
`role_ref_id`	INT	FK to role_definition_header_all.id
`previous_status`	INT	Previous status code
`new_status`	INT	New status code
`reason`	VARCHAR(500)	Reason for status change
`changed_by`	INT	User ID who performed the transition
`changed_by_ref_id`	INT	FK to user_header_all.id
`changed_on`	DATETIME	Timestamp of the transition
`remarks`	TEXT	Additional remarks
`status`	INT	1=active, 2=inactive, 3=deleted
`created_on`	DATETIME	Record creation timestamp
`modified_on`	DATETIME	Record modification timestamp

■ INSERT when	Every time the status column in role_definition_header_all changes. Record previous_status, new_status, and datetime.
■ UPDATE when	Never update life cycle rows — append only.

3. PRODUCTION RULES APPLIED

The following 36 rules from the AI Database Rule Framework were applied during schema generation.

Rule ID	Rule Name	Category	Priority
1	Naming Taxonomy	naming	CRITICAL
2	Two-ID System	identity	CRITICAL
9	Centralised ID Registry	identity	CRITICAL
7	Indian GST Compliance	taxation	CRITICAL
98	Three-Table State Machine for Approvals	workflow	HIGH
11	Flat Approval Chains	workflow	HIGH
4	Life Cycle Tracking	workflow	HIGH
45	Temp-to-Permanent Promotion	architecture	HIGH
28	old_ Value Preservation Mechanics	audit	HIGH
20	Soft Delete Pattern	data_management	HIGH
81	Generic Approval Engine	workflow	HIGH
3	Three-Layer Data Preservation	audit	HIGH
30	Timestamp Column Naming	naming	HIGH
18	Comprehensive Audit Logging	audit	HIGH
39	Snapshot + Event Log Dual Table	architecture	HIGH
36	Prefixed Column Naming for JOIN-Heavy Schemas	naming	MEDIUM
95	Algorithm Parameters as Schema Columns	configuration	MEDIUM
92	Flat Tables for Fixed Third-Party API Responses	domain	MEDIUM
10	Infrastructure Monitoring via Schema	monitoring	MEDIUM
48	Parallel edited_* Correction Tables	architecture	MEDIUM
101	UI Process Registry in Schema	monitoring	MEDIUM
40	Resume Pointer on Session Entities	architecture	MEDIUM
8	Status Integer Convention	convention	MEDIUM
43	Deactivation Timestamp Column	audit	MEDIUM
60	Date-Bounded Configuration Records	temporal	MEDIUM
27	Denormalised Running Balance	financial	HIGH
29	Decimal Money Fields	financial	CRITICAL
51	Failed Transactions in Separate Table	financial	HIGH
57	Pre-Computed Analytics at Write Time	financial	HIGH
103	Multi-Tier Transaction Consolidation	financial	HIGH
38	Aggregate Counters on Parent Rows	performance	HIGH
52	DB-Backed Async Job Queue	operations	HIGH

54	Monthly Calendar Table Pattern	calendar	HIGH
94	Operational State on Reusable Entity Row	architecture	MEDIUM
31	Named FK Constraints	constraints	HIGH
32	Named Indexes	constraints	HIGH

4. DEVELOPER NOTES

Business ID Generation	All business IDs (student_id, fee_id, etc.) must be generated from unique_id_header_all. Insert a row to register each entity type with its prefix. Increment last_id on every new record. Never use the auto-increment 'id' column as a business identifier.
Foreign Key References	All foreign keys must reference the integer 'id' PRIMARY KEY column, not the business ID (varchar) column. The business ID is for display only. Join tables using integer id columns for performance.
Archive Table Usage	Copy the full row to the corresponding _archive_all table BEFORE making any UPDATE to a _header_all table. Archive tables store the history of every change. Never DELETE from _header_all — use status instead.
Life Cycle Table Usage	Insert a row into _life_cycle_all every time the status column changes. Store previous_status, new_status, and the datetime of the change. This enables full audit trail of every state transition.
GST Handling	Always store CGST and SGST amounts separately. Never combine them into a single tax column. Calculate at write time and store — never compute at read time. Current rate: CGST 9% + SGST 9% = 18% total (verify with your accountant for current applicable rates).
Closing Balance	The closing_balance column on transaction tables must be updated atomically with the transaction insert. Use a database transaction (BEGIN/COMMIT) to ensure the balance and the transaction record are always consistent. Never update balance separately.
Status Convention	Status 1 = active/success. Status 2 = inactive/failure. Status 3 = deleted/cancelled. Never hard DELETE rows. Set status = 2 or 3 and use soft delete everywhere.
Index Usage	All foreign key columns are indexed. Use the named indexes in your WHERE clauses. Avoid SELECT * in production — always select specific columns to use covering indexes effectively.